ASSIGNMENT 1

CO1: *Java fundamentals and object-oriented programming – data types, operators, control statements, arrays, classes and objects, constructors, static members, inheritance, method overriding, polymorphism, abstract classes, interfaces, packages, access modifiers and encapsulation*

Assignment Title: Design, Implement and Validate an Object-Oriented Model for a Campus Smart-Parking System (Total Marks - 50)

Nature of Assignment: Individual **Duration:** 1 Week

Submission Date: _____

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA09-<RegisterNumber>-Assignment1** and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the Java source code, input files and output screenshots.
- Include a README.md with your name, register number, Java version, the steps to compile and run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: https://github.com/Pavithra16780/CSA09-192472155-Assignment1.git

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Scenario — Campus Smart-Parking System

The university campus has a multi-level parking facility used by students, staff and visitors. The facility has several floors, and each floor has a fixed number of numbered slots. Vehicles entering the campus may be two-wheelers, cars or electric vehicles (EVs). EVs may also use charging slots, which are charged separately.

When a vehicle enters, the system must allocate a suitable free slot, issue a parking ticket with a unique ticket number, and record the entry time. When the vehicle leaves, the system must release the slot and compute the parking fee. Fees depend on the vehicle type and the duration of parking. Staff and students holding a valid monthly pass park free of charge, while visitors pay the full fee.

The facility may contain:

1 floor × 20 slots → 3 floors × 50 slots → 5 floors × 100 slots

You are required to analyse the problem, design an object-oriented model for it, implement the model in Java and validate it. Your design must remain correct and easy to extend as the facility and its rules grow.

Design Challenge:

The university now wants to add new vehicle categories and a new pricing rule without rewriting the classes that already work.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Understand the Problem and Identify the Objects (4 Marks)

1.1 Inputs and Outputs

Inputs	**Outputs**
Vehicle registration number and vehicle category	Allocated floor and slot number
User category: student, staff or visitor	Unique parking ticket
Pass type and validity	Entry time and exit time record
Entry time and exit time / parking duration	Parking fee and payment status
Floor and slot configuration	Updated occupancy and available-slot information

1.2 Candidate Classes and Responsibilities

Candidate class	**Why it is required / responsibility**
ParkingFacility	Represents the complete parking area; owns floors/slots and coordinates allocation and release.
ParkingSlot	Represents one numbered slot; stores whether it is free and the vehicle/ticket occupying it.
Vehicle	Common parent type; stores registration number and defines fee behaviour.

TwoWheeler, Car, ElectricCar	Represent vehicle categories with their own hourly rates or charging capability.
ParkingTicket	Stores unique ticket number, vehicle, allocated slot, entry time and exit details.
ParkingUser / Student / Staff / Visitor	Represents the person using the facility and their pass eligibility.
ParkingPass / MonthlyPass / OneDayPass	Represents pass validity and fee-waiver rules.
PricingRule	Separates fee policy from vehicle and parking-slot details.
ParkingService	Coordinates entry, exit, duplicate checking, ticket lookup and billing.

1.3 Nouns and Verbs

Candidate nouns include campus, floor, slot, vehicle, two-wheeler, car, electric vehicle, ticket, user, student, staff, visitor, pass, fee and duration. These become candidate classes, fields or enumerated values.

Important verbs include enter, allocate, issue, record, leave, release, calculate, validate, charge and display. These become methods such as allocateSlot(), issueTicket(), recordEntry(), releaseSlot(), calculateFee() and validatePass().

1.4 Business Rules

A slot can contain at most one vehicle at a time.

A vehicle cannot be allocated to two slots simultaneously.

A ticket number must be unique and must never be reused.

A vehicle must have a valid registration number and supported category.

A slot index must be within the configured floor and slot limits.

A slot can be released only when it is occupied.

A negative parking duration or negative fee is invalid.

A valid monthly pass for a student or staff member makes the parking fee zero.

A visitor without a valid pass pays the calculated fee.

The system must report clearly when no suitable slot is available.

1.5 Assumptions

The initial facility is configured through an array containing the number of slots on each floor. Slot numbering starts at zero internally, although the user interface may display slot numbers starting at one. The first suitable free slot is allocated. The system uses a single active ticket for each parked vehicle.

1.6 Alternative: Objects vs Parallel Arrays

In an object-oriented design, each Vehicle object keeps related data and behaviour together. Searching by registration number still requires a linear scan if vehicles are stored in an ordinary array, so the time complexity is O(n). The object approach is easier to extend: a new field such as colour is added once to Vehicle.

With parallel arrays such as regNo[], type[] and entryTime[], the same vehicle's details are stored at the same index in every array. A search is also O(n), but all arrays must remain aligned. Adding a new attribute requires another array and updates to every insertion, deletion and copying operation.

Measure	Vehicle objects	Parallel arrays
Find registration number	O(n) linear scan	O(n) linear scan
Add one attribute	Add a field/accessor in Vehicle or a subclass	Create another array and maintain matching indexes
Space per vehicle	O(1) fields plus object/reference overhead	O(1) values across arrays; separate array overhead
Maintenance	Related data and behaviour stay together	Index alignment can cause errors

Conclusion: Vehicle objects are preferred because the system has different vehicle behaviours and will grow with new categories.

TASK 2 – Choose the Data Representation (3 Marks)

Data item	Java type	Reason
Registration number	String	Contains letters and digits; arithmetic is not required.
Slot number / floor number	int	Whole-number indexes.
Entry and exit time	LocalDateTime	Stores date and time accurately.
Duration	long or int minutes	Whole number of elapsed minutes; long supports large values.
Hourly rate / fee	double or BigDecimal	Money may contain decimal values; BigDecimal is

		preferable for production billing.
Occupancy	boolean or null reference	Indicates whether a slot is occupied.
Ticket number	long	Supports a large unique sequence.
Pass validity	boolean and LocalDate dates	Represents validity and expiry.

2.1 Array Representation

A jagged two-dimensional array, String[][] slots, is suitable because each floor can have a different number of slots. The outer array represents floors; each inner array represents the slots on that floor. A null value means that the slot is free.

```
String[][] slots = new String[slotsPerFloor.length][];

for (int floor = 0; floor < slotsPerFloor.length; floor++) {

   slots[floor] = new String[slotsPerFloor[floor]];

}
```

2.2 Type Conversion and Precision

The number of billable hours is the ceiling of minutes divided by 60. Integer division alone truncates the fraction. For example, 130 / 60 gives 2, but 130 minutes must be billed as 3 started hours.

```
int minutes = 130;

int wrongHours = minutes / 60;            // 2: incorrect for started-hour billing
```

int billableHours = (minutes + 59) / 60; // 3: correct for non-negative minutes

double rate = 30.0;

double fee = billableHours * rate;   // Rs 90.0

Avoid casting after integer division: (double)(minutes / 60) still produces 2.0 because the division has already lost the fraction. Use (double) minutes / 60.0 when fractional hours are actually required, or use the integer ceiling formula above.

2.3 Input Method

Scanner is appropriate for a beginner-friendly console application because it can read strings, integers and menu choices interactively. Command-line arguments are useful for automated runs, but they are less convenient for a user entering several values.

2.4 Alternative Representations and Complexity

Representation	**Slot lookup**	**Different slot count per floor**	**Space**
Jagged String[floor][slot]	O(1) when floor and slot are known; O(F×S) to scan all slots	Naturally supported	O(total slots)
Flat one-dimensional array	O(1) by computed index when every floor has equal slots; otherwise prefix offsets are needed	Needs offset table or mapping	O(total slots)
ArrayList<String[]>	O(1) indexed access; O(F×S) to scan all slots	Naturally supported with one list per floor	O(total slots) plus list overhead

For unequal floor sizes, the jagged array or ArrayList<String[]> is safer than the simple formula floor × slotsPerFloor + slot. The simple formula assumes the same number of slots on every floor.

TASK 3 – Design the Control Logic (4 Marks)

3.1 Slot Allocation Pseudocode

```
ALGORITHM AllocateSlot(vehicleNo)
    FOR each floor from 0 to numberOfFloors - 1
        FOR each slot from 0 to slotsOnFloor - 1
            IF slot is empty THEN
                store vehicleNo in slot
                RETURN floor and slot
            END IF
        END FOR
    END FOR
    RETURN NO_SLOT_AVAILABLE
END ALGORITHM
```

3.2 Fee Computation Pseudocode

```
ALGORITHM CalculateFee(vehicle, minutes, pass)
    IF minutes < 0 THEN
        RETURN INVALID_DURATION
    END IF
```

```
    IF pass is valid and user is eligible THEN
        RETURN 0
    END IF
    hours = (minutes + 59) DIV 60
    fee = hours * vehicle hourly rate
    IF fee is greater than daily maximum THEN
        fee = daily maximum
    END IF
    RETURN fee
END ALGORITHM
```

3.3 Use of break, continue and return

return immediately ends allocation after the first suitable slot is found.

break can stop an inner or outer search when a required condition is met.

continue skips an unsuitable slot, such as a slot reserved for another vehicle category.

If all floors are searched without success, allocate() returns null.

3.4 Complete the Code – ParkingFacility.java

The following program completes both TODO sections and preserves the supplied main() method.

```
public class ParkingFacility {
    private final String[][] slots;
```

```
public ParkingFacility(int[] slotsPerFloor) {
    slots = new String[slotsPerFloor.length][];
    for (int f = 0; f < slotsPerFloor.length; f++) {
        if (slotsPerFloor[f] < 0)
            throw new IllegalArgumentException("Negative slot count");
        slots[f] = new String[slotsPerFloor[f]];
    }
}

public int[] allocate(String vehicleNo) {
    if (vehicleNo == null || vehicleNo.isBlank())
        throw new IllegalArgumentException("Invalid vehicle number");

    for (int f = 0; f < slots.length; f++) {
        for (int s = 0; s < slots[f].length; s++) {
            if (slots[f][s] == null) {
                slots[f][s] = vehicleNo;
                return new int[] {f, s};
            }
        }
```

```
    }
    return null;
}

public boolean release(int floor, int slot) {
    if (floor < 0 || floor >= slots.length)
        return false;
    if (slot < 0 || slot >= slots[floor].length)
        return false;
    if (slots[floor][slot] == null)
        return false;

    slots[floor][slot] = null;
    return true;
}

public static void main(String[] args) {
    ParkingFacility p = new ParkingFacility(new int[]{2, 1});
    System.out.println(java.util.Arrays.toString(p.allocate("TN01AB1111")));
    System.out.println(java.util.Arrays.toString(p.allocate("TN02CD2222")));
```

```
        System.out.println(java.util.Arrays.toString(p.allocate("TN03EF3333")));
        System.out.println(java.util.Arrays.toString(p.allocate("TN04GH4444")));
        System.out.println(p.release(0, 1) + " " + p.release(0, 1) + " "
                + p.release(5, 0));
        System.out.println(java.util.Arrays.toString(p.allocate("TN04GH4444")));
    }
}
```

3.5 Expected Output

[0, 0]

[0, 1]

[1, 0]

null

true false false

[0, 1]

3.6 Alternative: Free-Slot Stack

Instead of scanning every slot, maintain a stack of free slot identifiers for each floor and a free-slot count. Allocation pops one free slot; release pushes the slot back. With a suitable stack, allocate() and release() take O(1) time. The stack requires O(T) extra space, where T is the total number of slots. The scan approach needs no separate free-slot stack but can take O(T) time per operation.

Approach	allocate()	release()	Extra space
Scan all slots	O(T) worst case	O(1) after validating indexes	O(1) extra
Stack of free slots per floor	O(1)	O(1)	O(T) for free-slot entries

TASK 4 – Design Classes, Objects and Constructors (4 Marks)

4.1 Class Fields and Methods

Class	Example fields	Example methods
Vehicle	regNo	getRegNo(), ratePerHour(), calculateFee()
ParkingTicket	ticketNo, vehicle, floor, slot, entryTime	getDuration(), closeTicket(), toString()
ParkingFacility	slots, floor configuration	allocate(), release(), availableCount()
ParkingUser	userId, name, userType	hasValidPass(), getUserType()
ParkingService	facility, active tickets	vehicleEntry(), vehicleExit(), findTicket()

4.2 Constructors

A default constructor creates an object with default values. A parameterised constructor accepts the values needed to create a meaningful object. Overloaded constructors provide more than one valid way to create the same class.

```
class ParkingTicket {

    private final long ticketNo;

    private final String regNo;

    private final int floor;

    private final int slot;



    ParkingTicket(long ticketNo, String regNo) {

        this(ticketNo, regNo, -1, -1);

    }



    ParkingTicket(long ticketNo, String regNo, int floor, int slot) {

        if (ticketNo <= 0 || regNo == null || regNo.isBlank())

            throw new IllegalArgumentException("Invalid ticket data");

        this.ticketNo = ticketNo;

        this.regNo = regNo;

        this.floor = floor;

        this.slot = slot;
```

```
    }
}
```

The this keyword refers to the current object. Constructor chaining uses this(...) as the first statement of a constructor to reuse validation and initialisation logic.

4.3 Copy Constructor

A copy constructor is optional. It may be useful when creating a snapshot or duplicate display object from an existing ticket. However, a copied ticket must not receive the same active ticket number if ticket numbers are required to be unique. A new ticket should be issued through the ticket-number generator.

4.4 Stack and Heap

A local reference variable such as ticket is typically stored in a method's stack frame. The object created with new ParkingTicket(...) is stored on the heap. The reference points to the object. When no reachable reference points to an object, it becomes eligible for garbage collection.

4.5 Alternative: Setters and Static Factories

With a default constructor followed by setter calls, an object may temporarily contain incomplete or invalid data. A parameterised constructor validates all required fields at creation. A static factory such as ParkingTicket.create(...) can validate input and return a valid object or report an error.

Approach	**Validity**	**Creation time**	**Space for n tickets**
Parameterised constructor	Strong: validates at creation	O(1) per ticket	O(n)
Default constructor + setters	May be temporarily invalid	O(1) per ticket, plus setter calls	O(n)

Static factory	Strong if factory validates	O(1) per ticket	O(n)

TASK 5 – Apply Static Members and the Object Life Cycle (3 Marks)

5.1 Static Members

A ticket counter belongs to the ParkingTicket class because all tickets must share one sequence. If it were an instance variable, each ticket object could start its own counter and generate duplicate numbers.

```
class TicketNumber {

   private static long nextNumber = 1;



   public static synchronized long generate() {

      return nextNumber++;

   }

}
```

A static method can generate ticket numbers or validate class-wide configuration. A static block is appropriate only when class-wide initialisation is required; it is not necessary for every application.

5.2 Object Life Cycle

A ticket object is created when a vehicle enters.

The active-ticket collection holds a reference to the ticket while the vehicle is parked.

At exit, the system records exit time, calculates the fee and releases the slot.

After the ticket is removed from active records and no other object refers to it, it becomes eligible for garbage collection.

5.3 Passing and Returning Objects

Java passes object references by value. A method can receive a Vehicle reference, read its properties and call its methods. A method can return a ParkingTicket reference after successful allocation.

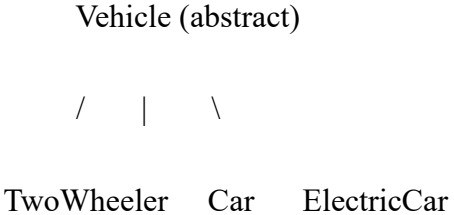
5.4 Alternative: Instance Counter per Facility

If the counter is an instance field of ParkingFacility, two facilities (north and south) can each start at 1 and issue the same ticket number. If ticket numbers must be globally unique, use one static counter or a central ticket-number service. If uniqueness is required only within each facility, use a facility identifier plus a local sequence, for example N-1001 and S-1001.

Design	**Counter storage for k facilities**	**Risk**
Static counter	One shared counter: O(1) space	Needs synchronisation for concurrent access
Instance counter	k counters: O(k) space	Numbers may collide across facilities unless facility ID is included

TASK 6 – Design the Inheritance Hierarchy (4 Marks)

6.1 Vehicle Hierarchy



(extends Car)

implements Chargeable

Vehicle is the superclass. TwoWheeler and Car use hierarchical inheritance from Vehicle. ElectricCar extends Car, producing multilevel inheritance. ElectricCar also implements the Chargeable interface.

Class	**Inherited features**	**Added / specialised features**
Vehicle	Common registration number and fee contract	Abstract ratePerHour()
TwoWheeler	Registration number and fee calculation	Two-wheeler hourly rate
Car	Registration number and fee calculation	Car hourly rate
ElectricCar	Vehicle and Car behaviour	Charging capability and chargingFee()

6.2 Use of super

super(regNo) calls the parent constructor. super.method() can call a parent implementation when the subclass wants to extend rather than fully replace it. The parent constructor executes before the child constructor.

6.3 ElectricCar Constructor Trace

Vehicle constructor

↓

Car constructor

↓

ElectricCar constructor

When new ElectricCar("TN03C3") is executed, the ElectricCar constructor calls super(...), which invokes the Car constructor. The Car constructor calls the Vehicle constructor. After Vehicle initialisation completes, control returns to Car and then ElectricCar.

6.4 Alternative: Composition

With composition, Vehicle would contain a VehicleType object that supplies the hourly rate. Adding a new type would require creating a new VehicleType implementation and configuring the vehicle with it. Inheritance expresses an is-a relationship; composition expresses a has-a relationship and makes it easier to change pricing behaviour at runtime.

Measure	**Inheritance**	**Composition**
Add a vehicle type	Usually add a subclass	Add a VehicleType implementation
Fee calculation	O(1) virtual method dispatch	O(1) delegated method call
Per-vehicle space	Common fields plus object header	Common fields plus reference; strategy object may be shared
Flexibility	Type behaviour tied to class hierarchy	Pricing strategy can be replaced independently

TASK 7 – Override and Specialise Behaviour (4 Marks)

7.1 Method Overriding

Each vehicle category can provide its own ratePerHour() implementation. The common calculateFee() method uses the rate returned by the actual object. The @Override annotation asks the compiler to verify that the method correctly overrides a parent method.

```
abstract class Vehicle {

    private final String regNo;

    protected Vehicle(String regNo) { this.regNo = regNo; }

    public String getRegNo() { return regNo; }

    protected abstract double ratePerHour();



    public double calculateFee(int minutes) {

        if (minutes < 0) throw new IllegalArgumentException("Negative duration");

        int hours = (minutes + 59) / 60;

        return hours * ratePerHour();

    }

}



class TwoWheeler extends Vehicle {

    TwoWheeler(String regNo) { super(regNo); }

    @Override protected double ratePerHour() { return 10.0; }
```

```
}


class Car extends Vehicle {
    Car(String regNo) { super(regNo); }
    @Override protected double ratePerHour() { return 30.0; }
}
```

7.2 Rules of Overriding

The method name and parameter list must match the inherited method signature.

The return type must be the same or a permitted covariant return type.

The overriding method cannot reduce the access level.

A final method cannot be overridden; a final class cannot be subclassed.

Static methods are hidden, not overridden.

7.3 Object Methods

toString() should return useful vehicle details. equals() should compare the identity rule chosen by the application, such as registration number. hashCode() must use the same fields as equals(). If two vehicles with the same registration number are equal, they must return the same hash code.

```
@Override
public boolean equals(Object obj) {
    if (this == obj) return true;
    if (!(obj instanceof Vehicle)) return false;
```

```
    Vehicle other = (Vehicle) obj;

    return regNo.equals(other.regNo);

}


@Override

public int hashCode() {

    return regNo.hashCode();

}
```

For a real parking system, the equality rule should be reviewed: two records with the same registration number may represent the same physical vehicle, but an active ticket is a separate parking event.

7.4 Alternative: switch on Type

A single calculateFee() method could inspect a type field using switch and select a rate. The lookup remains O(1) for a fixed number of enum cases, but every new vehicle type requires editing the central switch. With overriding, adding a new subclass can avoid changing tested fee-calculation code.

Approach	**Time per fee calculation**	**Change when new type is added**
Overriding	O(1) dispatch and calculation	Add a subclass / implementation
switch on type	O(1) for fixed enum switch; O(k) for sequential checks	Modify central switch and retest it

TASK 8 – Apply Polymorphism (4 Marks)

8.1 Compile-Time Polymorphism

Method overloading is compile-time polymorphism. The compiler selects the method by examining the argument types and number of arguments.

```
double calculateFee(int minutes) { /* calculation */ return 0; }

double calculateFee(java.time.LocalTime in, java.time.LocalTime out) {
    long minutes = java.time.Duration.between(in, out).toMinutes();
    return calculateFee((int) minutes);
}
```

8.2 Runtime Polymorphism

A Vehicle[] array can hold references to TwoWheeler, Car and ElectricCar objects. This is upcasting. When v.calculateFee(130) is called, Java chooses the overridden implementation at runtime according to the actual object.

8.3 Complete the Code – FeeDemo.java

```
interface Chargeable {
    double chargingFee(int kWh);
}


abstract class Vehicle {
    private final String regNo;
```

```
protected Vehicle(String regNo) {
    if (regNo == null || regNo.isBlank())
        throw new IllegalArgumentException("Invalid registration number");
    this.regNo = regNo;
}

public String getRegNo() { return regNo; }
protected abstract double ratePerHour();

public double calculateFee(int minutes) {
    if (minutes < 0) throw new IllegalArgumentException("Negative minutes");
    int hours = (minutes + 59) / 60;
    return hours * ratePerHour();
}

@Override public boolean equals(Object o) {
    return o instanceof Vehicle
            && regNo.equals(((Vehicle) o).regNo);
}
```

```
    @Override public int hashCode() { return regNo.hashCode(); }
}

class TwoWheeler extends Vehicle {
    TwoWheeler(String regNo) { super(regNo); }
    @Override protected double ratePerHour() { return 10; }
}

class Car extends Vehicle {
    Car(String regNo) { super(regNo); }
    @Override protected double ratePerHour() { return 30; }
}

class ElectricCar extends Car implements Chargeable {
    ElectricCar(String regNo) { super(regNo); }
    @Override public double chargingFee(int kWh) {
        if (kWh < 0) throw new IllegalArgumentException("Invalid kWh");
        return 12.0 * kWh;
    }
}
```

```
public class FeeDemo {

    public static void main(String[] args) {

        Vehicle[] parked = {

            new TwoWheeler("TN01A1"),

            new Car("TN02B2"),

            new ElectricCar("TN03C3")

        };



        for (Vehicle v : parked) {

            double fee = v.calculateFee(130);

            if (v instanceof Chargeable c)

                fee += c.chargingFee(10);

            System.out.println(v.getClass().getSimpleName() + " "

                    + v.getRegNo() + " -> Rs " + fee);

        }

        System.out.println(new Car("TN02B2")

                .equals(new ElectricCar("TN02B2")));

    }

}
```

8.4 Expected Output

TwoWheeler TN01A1 -> Rs 30.0

Car TN02B2 -> Rs 90.0

ElectricCar TN03C3 -> Rs 210.0

true

The last line is true because the supplied equals() method compares only registration numbers. Both objects have registration number TN02B2. This is correct only if registration number alone defines Vehicle equality in the application. A production system may prefer to keep vehicle identity separate from ticket identity.

8.5 Dispatch Trace

Expression	**Value**
Declared reference type	Vehicle
Actual object type	ElectricCar
Called method	calculateFee(130)
Rate method selected at runtime	ElectricCar inherits Car.ratePerHour(), which returns 30
Parking fee	3 started hours × Rs 30 = Rs 90
Charging fee	10 kWh × Rs 12 = Rs 120
Total	Rs 210

8.6 Alternative: instanceof and Casting

The alternative is to test the runtime type using a chain of instanceof checks and cast the object to the relevant subclass. With k categories, a sequential chain may perform up to k checks, giving O(k) type selection. Dynamic dispatch is conceptually one virtual method

selection, commonly treated as O(1). The instanceof approach becomes longer and must be edited whenever a category is added.

TASK 9 – Introduce Abstraction (4 Marks)

9.1 Abstract Class and Interface

Vehicle should be abstract because a generic vehicle does not have one universal hourly rate. It can store shared state such as registration number, provide common fee logic and declare ratePerHour() as abstract.

Chargeable should be an interface because charging is a capability that only some vehicles support. An ElectricCar can extend Car and implement Chargeable. A class can implement more than one interface, allowing multiple capabilities without multiple class inheritance.

9.2 Why Use Both?

Abstract class	**Interface**
Can contain instance fields and constructors	Cannot be instantiated and does not provide ordinary per-object constructor state
Shares common state and implementation	Defines a capability or contract
A class can extend only one class	A class can implement multiple interfaces
Useful for Vehicle	Useful for Chargeable or Payable

9.3 Default and Static Interface Methods

```
interface Chargeable {

    double chargingFee(int kWh);
```

```
    default boolean supportsFastCharging() {
        return false;
    }

    static int standardRatePerKWh() {
        return 12;
    }
}
```

A default method supplies a common optional behaviour that implementing classes may override. A static interface method is called using the interface name, for example Chargeable.standardRatePerKWh().

9.4 Alternative: Vehicle Interface and Chargeable Abstract Class

If Vehicle became an interface, it could define method contracts and default methods, but it could not use an ordinary constructor to initialise a private registration-number instance field for every vehicle. Each implementing class would need to store that state itself. If Chargeable became an abstract class, a vehicle could not extend both Car and Chargeable because Java supports only one direct superclass. The design would lose the clean combination of a vehicle class hierarchy and an independent charging capability.

Method-call time is effectively O(1) in both designs. The main difference is modelling flexibility and code organisation, not asymptotic execution time.

TASK 10 – Enforce Encapsulation through Packages and Access Modifiers (3 Marks)

10.1 Package Structure

parking/

model/

Vehicle.java

Car.java

TwoWheeler.java

ElectricCar.java

ParkingTicket.java

ParkingPass.java

service/

ParkingService.java

PricingRule.java

facility/

ParkingFacility.java

ParkingSlot.java

app/

Main.java

10.2 Access Modifiers

Modifier	**Use in this design**
private	Fields such as regNo, ticketNo and slot occupancy; protects internal state.

default (package-private)	Helper classes or methods used only within one package.
protected	Subclass extension points such as ratePerHour().
public	Methods that form the supported API, such as getRegNo() and vehicleEntry().

10.3 Validating Getters and Setters

Getters return controlled information. Setters validate values before changing state. Immutable values such as registration number and ticket number should preferably be final and have no setter.

```
public void setDurationMinutes(int minutes) {
    if (minutes < 0)
        throw new IllegalArgumentException("Duration cannot be negative");
    this.durationMinutes = minutes;
}


public void setSlotNumber(int slotNumber) {
    if (slotNumber < 0)
        throw new IllegalArgumentException("Invalid slot number");
    this.slotNumber = slotNumber;
}
```

10.4 Wrapper Classes and Autoboxing

Integer, Double and Long are wrapper classes for primitive values. Autoboxing converts a primitive to its wrapper, and unboxing converts it back. Avoid comparing Integer objects using == because it may compare object references rather than numeric values. Use intValue(), equals(), or compareTo() as appropriate.

10.5 Alternative: Public Fields

If fields were public, any calling class could assign an invalid duration, negative fee or out-of-range slot. Validation would need to be repeated at every call site. With a setter, validation is centralised. A simple validation check takes O(1) time per update, but duplicated checks increase maintenance work and the number of places where defects can occur.

Design	Validation cost per update	Maintenance
Private fields + validating setter	O(1)	One central validation point
Public fields + caller validation	Depends on caller; each check often O(1)	Same rule repeated in many locations

TASK 11 – Implement, Test and Defend the Design (5 Marks)

11.1 Implementation Plan

The system is divided into model, facility and service responsibilities. Vehicle subclasses determine vehicle-specific rates. ParkingFacility manages floor/slot occupancy. ParkingTicket stores the entry record. ParkingService coordinates entry and exit, checks duplicate registrations and applies pass eligibility and pricing rules.

11.2 Test Cases

Test	Input / condition	Expected result
Normal entry	Two-wheeler, one free slot	Ticket created and slot allocated
Normal exit	Valid active ticket	Slot released and fee returned
Last free slot	Only one slot remains	Vehicle receives that slot
Full facility	No free suitable slot	Entry rejected with no-slot message
Zero-minute stay	Duration = 0	0 billable hours and Rs 0
Fee boundary	60 minutes at Rs 30/hour	1 hour; Rs 30
Just above boundary	61 minutes at Rs 30/hour	2 hours; Rs 60
Unknown type	Unsupported category text	Validation error; no slot occupied
Duplicate registration	Same registration already parked	Second entry rejected
Invalid slot	Floor or slot index out of range	Release returns false
Monthly pass	Eligible student/staff with valid pass	Parking fee Rs 0
Visitor without pass	Visitor pays normal rate	Fee calculated normally

11.3 Hand Calculation of Daily Collection

Example day: one two-wheeler parks for 90 minutes, one car parks for 130 minutes, and one eligible staff member with a valid monthly pass parks for 45 minutes.

Vehicle	**Calculation**	**Fee**
Two-wheeler	2 started hours × Rs 10	Rs 20
Car	3 started hours × Rs 30	Rs 90
Staff with valid pass	Fee waived	Rs 0
Total collection	20 + 90 + 0	Rs 110

The program's total should be Rs 110 for this example when the stated rates and pass rule are applied.

11.4 Design Evaluation

Encapsulation is followed by keeping data private and validating updates.

Inheritance is used to share common vehicle state and specialise vehicle categories.

Polymorphism allows fee calculation through Vehicle references.

Abstraction separates common vehicle behaviour from category-specific rates.

A limitation of the simple array-based version is that searching for an active ticket takes O(n) time as the number of parked vehicles grows.

11.5 Active Ticket Storage Alternatives

Structure	**Find ticket by registration**	**Insertion / removal**	**Space**
Array	O(n)	Insertion may be O(1) if free index	O(n)

		known; removal O(1) after search	
ArrayList<ParkingTicket>	O(n) search; removal O(n) due to shifting	Append amortised O(1); remove by index O(n)	O(n)
HashMap<String, ParkingTicket>	Average O(1), worst-case O(n)	Average O(1) put/remove	O(n)

For a large facility, HashMap<String, ParkingTicket> keyed by registration number is a suitable choice for quick exit lookup. A separate slot structure is still needed to manage physical occupancy.

TASK 12 – Re-evaluate and Adapt the Design (8 Marks)

A. Impact Analysis (2 Marks)

The changed requirements introduce e-bikes, delivery vans, visitor one-day passes and a five-floor facility with unequal slot counts. E-bikes have a flat fee. Delivery vans are charged at double the car rate during 9 AM–5 PM. Visitors may purchase a one-day pass.

Change	**Affected part**	**Required response**
E-bike category	Vehicle hierarchy and pricing	Add E-Bike type with flat-fee rule
Delivery van	Vehicle hierarchy and pricing	Add DeliveryVan and time-window pricing rule
Visitor one-day pass	Pass model and eligibility check	Add OneDayPass with date validity

Five floors with unequal slots	Facility storage	Use jagged array or list of per-floor slot arrays
New time-based rate	Pricing logic	Separate pricing rule from slot allocation

A jagged array is required because the number of slots differs by floor. A rectangular array would waste positions or fail to represent the exact configuration.

B. Evaluation of Original Design (2 Marks)

The original design can support new vehicle types through subclasses, but if every pricing rule is embedded inside subclasses, time-based rules and pass exceptions can spread across many classes. This makes future changes harder. The original jagged-array representation remains suitable. The fee calculation should be modified to delegate to a PricingRule abstraction so that new policies can be added without rewriting allocation logic.

The best choice is MODIFY rather than completely redesign. Keep the tested Vehicle, ParkingFacility and ticket concepts; extend the vehicle and pass models; and move changing fee policies into separate pricing-rule classes.

C. Adaptation / Revised Class Structure (2 Marks)

Vehicle (abstract)

|-- TwoWheeler

|-- Car

| |-- ElectricCar implements Chargeable

|-- EBike

|-- DeliveryVan

ParkingPass (interface)

|-- MonthlyPass

|-- OneDayPass

PricingRule (interface)

|-- StandardPricingRule

|-- DeliveryVanTimePricingRule

|-- EBikeFlatPricingRule

ParkingFacility -> jagged array of ParkingSlot[] per floor

ParkingService -> entry, exit, ticket lookup and pass validation

12.1 Example Classes for New Categories

```
abstract class Vehicle {
    private final String regNo;
    protected Vehicle(String regNo) { this.regNo = regNo; }
    public String getRegNo() { return regNo; }
    public abstract double calculateFee(int minutes);
}

class EBike extends Vehicle {
    private static final double FLAT_FEE = 50.0; // sample rate
```

```
    EBike(String regNo) { super(regNo); }
    @Override public double calculateFee(int minutes) {
        if (minutes < 0) throw new IllegalArgumentException();
        return minutes == 0 ? 0.0 : FLAT_FEE;
    }
}

class DeliveryVan extends Vehicle {
    private static final double CAR_RATE = 30.0; // sample rate
    DeliveryVan(String regNo) { super(regNo); }

    public double calculateFee(int minutes, boolean withinNineToFive) {
        if (minutes < 0) throw new IllegalArgumentException();
        int hours = (minutes + 59) / 60;
        double rate = withinNineToFive ? 2 * CAR_RATE : CAR_RATE;
        return hours * rate;
    }

    @Override public double calculateFee(int minutes) {
        // The caller should use the time-aware overload when entry/exit
```

```
        // overlaps the 9 AM–5 PM charging window.
        int hours = (minutes + 59) / 60;
        return hours * CAR_RATE;
    }
}
```

The delivery-van time-window calculation needs actual entry and exit timestamps in a full implementation. The boolean overload above demonstrates the rate decision; production code should calculate the minutes that overlap 9 AM–5 PM and apply the doubled rate only to that portion.

12.2 One-Day Pass

```
import java.time.LocalDate;

interface ParkingPass {
    boolean isValidOn(LocalDate date);
}

class OneDayPass implements ParkingPass {
    private final LocalDate validDate;

    OneDayPass(LocalDate validDate) {
        if (validDate == null) throw new IllegalArgumentException();
```

```
        this.validDate = validDate;
    }

    @Override
    public boolean isValidOn(LocalDate date) {
        return validDate.equals(date);
    }
}
```

12.3 Unequal Floor Configuration

```
int[] slotsPerFloor = {20, 35, 40, 50, 60};
ParkingSlot[][] floors = new ParkingSlot[slotsPerFloor.length][];
for (int f = 0; f < slotsPerFloor.length; f++) {
    floors[f] = new ParkingSlot[slotsPerFloor[f]];
    for (int s = 0; s < slotsPerFloor[f].length; s++) {
        floors[f][s] = new ParkingSlot(f, s);
    }
}
```

The numbers above are illustrative configuration values; the changed requirement specifies five floors with different slot counts but does not prescribe the exact count for each floor.

D. Evidence-Based Defence (2 Marks)

Evidence item	Defence
Existing classes changed	Vehicle may be extended with a pricing strategy; ParkingFacility keeps its jagged-floor structure; pass validation is extended.
Ease of adding another type	Create a new Vehicle subclass and pricing rule rather than changing allocation code.
New-category tests	E-bike flat fee, delivery van during 9 AM–5 PM, delivery van outside that period, and one-day pass on valid/invalid dates must be tested.
Trade-off	More small classes and interfaces are introduced, but pricing rules become easier to test and extend.

12.4 New Requirement Test Cases

Test	Expected result
E-bike parks for 30 minutes	Sample flat fee Rs 50
E-bike parks for 5 hours	Sample flat fee remains Rs 50
Delivery van for 1 hour within 9 AM–5 PM	2 × Rs 30 = Rs 60
Delivery van for 1 hour outside 9 AM–5 PM	Rs 30
Visitor one-day pass used on its valid date	Parking fee waived according to pass policy
Visitor one-day pass used on next date	Pass invalid; normal fee applies

Unequal floors configured	Every floor has exactly its configured number of slots

Final decision: MODIFY the original design. Retain the object-oriented vehicle model, ticket concept and jagged array. Add new vehicle subclasses, a pass abstraction and separate pricing rules. This minimises changes to tested code while meeting the new requirements.

MARK DISTRIBUTION

Task	**Bloom's Level**	**Marks**
Task 1 – Problem Analysis & Objects	K4 – Analyse	4
Task 2 – Data Representation	K4 – Analyse	3
Task 3 – Control Logic & Pseudocode	K4 – Analyse	4
Task 4 – Classes, Objects & Constructors	K4 – Analyse	4
Task 5 – static Members & Object Life Cycle	K4 – Analyse	3
Task 6 – Inheritance Hierarchy	K4 – Analyse	4
Task 7 – Method Overriding	K4 – Analyse	4
Task 8 – Polymorphism	K4 – Analyse	4

Task 9 – Abstraction	K4 – Analyse	4
Task 10 – Packages & Encapsulation	K4 – Analyse	3
Task 11 – Implementation, Testing & Defence	K5 – Evaluate	5
Subtotal		**42**
Task 12 – Day-4 Design Challenge	K5 – Evaluate	**8**
TOTAL		**50**